Chapter 9 — DDL, DML and Constraints: Building Tables and Changing Data Safely

So far the book has only *read* data. This chapter is about creating the structures that hold data and changing what is in them:

- **DDL — Data Definition Language**: statements that define structure: `CREATE`, `ALTER`, `DROP`, `TRUNCATE` (and `CREATE INDEX`).
- **Keys and constraints**: rules attached to tables — primary and foreign keys, `UNIQUE`, `CHECK`, `NOT NULL`, `DEFAULT` — enforced on every change, whichever application writes.
- **DML — Data Manipulation Language**: statements that change rows: `INSERT`, `UPDATE`, `DELETE`, plus PostgreSQL's "insert or update" (`ON CONFLICT`) and `RETURNING`.
- **Backup and restore**: copying a database and getting data back after a mistake — run for real with `pg_dump` and `pg_restore` (PostgreSQL's command-line programs that write a database out to a file and load it back in).

Designing tables (entities, relationships, normal forms — sets of rules for keeping each fact in only one place) is chapter 10 and chapter 11; here we focus on the statements.

What you'll learn

#	Topic	You will be able to…
9.1	Data types	pick the right type for numbers, text, dates, booleans, and know what each rejects
9.2	`CREATE`	create databases, schemas, tables (also from queries and as copies), temporary tables
9.3	Keys and constraints	explain super/candidate/primary/alternate/foreign/composite/unique keys and use every constraint, including `ON DELETE` actions
9.4	Generated keys	use identity columns, sequences, `serial`, UUIDs and generated columns
9.5	`ALTER`	add/rename/drop columns, change types, add constraints safely on live tables
9.6	`DROP` and `TRUNCATE`	remove tables or empty them, and compare with `DELETE`
9.7	Indexes (brief)	create/drop indexes; clustered vs non-clustered
9.8	`INSERT`	insert single/multiple rows, defaults, `INSERT … SELECT`, `RETURNING`
9.9	`UPDATE`	update with conditions, expressions, `CASE`, other tables
9.10	`DELETE`	delete with conditions, other tables, soft deletes
9.11	Upsert and `MERGE`	`INSERT … ON CONFLICT`, and what `MERGE` (PostgreSQL 15+) looks like
9.12	Backup and restore	logical vs physical backups; `pg_dump` / `pg_restore`; other databases' commands

Setup. Examples run on the `company` sample database (chapter 1, section 1.11); each file in `code/` starts from a fresh copy and creates any extra tables it needs. Errors are shown on purpose — each `ERROR:` in an output block is the database refusing something.

9.1 Data types

What it is

Every column has a **data type** saying what kind of value it holds (whole number, decimal, text, date…). The type decides storage, which operations work, how values sort and — just as important — which values

are **rejected**. A type is the first and cheapest constraint.

The common types

Kind	PostgreSQL type	Holds	Use for	MySQL / SQL Server notes
Integer	`smallint` (2 bytes)	−32 768 … 32 767	small ranges: ranks, ratings	same names
	`integer` / `int` (4 bytes)	about ±2.1 billion	counts, most ids	same
	`bigint` (8 bytes)	about ±9.2 × 10^{18}	large ids, populations, byte counts	same
Exact decimal	`numeric(p, s)` / `decimal(p, s)`	*p* digits in total, *s* after the point	**money**, anything that must add up exactly	same
Approximate	`real`, `double precision`	binary floating point (fast but approximate)	measurements, science, ML (machine-learning) features	`FLOAT` / `DOUBLE`; SQL Server `float`
Fixed text	`char(n)`	exactly *n* characters, padded with spaces	fixed codes (rarely worth it)	same
Variable text	`varchar(n)`	up to *n* characters	text with a real maximum length	SQL Server: `nvarchar(n)` for Unicode (text in any alphabet)
	`text`	any length	most text in PostgreSQL	MySQL `TEXT` has limits and index restrictions
Date / time	`date`	a calendar day	birthdays, due dates	same
	`time`	time of day	opening hours	same
	`timestamp`	date + time, **no** time zone	local "wall clock" times	MySQL `DATETIME`
	`timestamptz`	an exact instant (stored in UTC — the world's reference clock — shown in the session's time zone; a *session* is your current connection)	event times, created_at	MySQL `TIMESTAMP` (converted to UTC); SQL Server `datetimeoffset`
Year only	(none) — use `smallint` + `CHECK`, or `date`			MySQL has `YEAR`
Boolean	`boolean`	true / false / NULL	flags	MySQL `BOOLEAN` = `TINYINT(1)`; SQL Server `bit`
Other useful	`uuid`, `jsonb`, arrays, `inet`, ranges		see chapter 14	

More on type conversion, casts and functions per type is in chapter 3.

Seeing the rules in action:

```sql
-- code/01_data_types.sql
SET TimeZone = 'UTC';   -- make timestamptz output the same on every machine
CREATE TABLE type_demo (
    qty        integer,
    views      bigint,
    rank_no    smallint,
    price      numeric(10,2),
    ratio      double precision,
    code       char(5),
    name       varchar(10),
    notes      text,
    born_on    date,
    starts_at  time,
    created    timestamp,
    created_tz timestamptz,
    active     boolean,
    birth_year smallint CHECK (birth_year BETWEEN 1900 AND 2100)  -- no YEAR type in PostgreSQL
);
INSERT INTO type_demo VALUES
 (3, 9000000000, 7, 19.999, 0.1::float8 + 0.2::float8, 'AB', 'Tara', 'long text ...',
  '1994-09-22', '09:30', '2025-06-01 10:00', '2025-06-01 10:00+05:30', true, 1994);

SELECT price, ratio, '[' || code || ']' AS code_padded, length(code) AS len_code,
       octet_length(code) AS bytes_code, created, created_tz
FROM type_demo;
```

Output:

```
 price |        ratio        | code_padded | len_code | bytes_code |       created       |       created_tz
-------+---------------------+-------------+----------+------------+---------------------+------------------------
 20.00 | 0.30000000000000004 | [AB]        |        2 |          5 | 2025-06-01 10:00:00 | 2025-06-01 04:30:00+00
(1 row)
```

What happened:

- `price` 19.999 was **rounded** to 2 decimal places → 20.00.
- `ratio`: 0.1 + 0.2 in binary floating point is not exactly 0.3 (`0.1::float8` is PostgreSQL's shorthand for a **cast** — converting a value to another type, here an 8-byte float) (the file also shows `0.1::numeric + 0.2::numeric = 0.3` is true, the `float8` version false). That's why money is never stored in `float`.
- `code` is stored padded to 5 characters (5 bytes), but PostgreSQL ignores a `char`'s trailing spaces, so `length` says 2 — confusing; prefer `text` / `varchar`.
- `created_tz`: 10:00 in India (+05:30) was stored as an instant and shown in the session time zone (UTC) as 04:30. **What the types reject:**

```sql
INSERT INTO type_demo (name) VALUES ('Christopher Columbus');
INSERT INTO type_demo (rank_no) VALUES (40000);
INSERT INTO type_demo (price) VALUES (123456789.00);
INSERT INTO type_demo (born_on) VALUES ('2025-02-30');
INSERT INTO type_demo (birth_year) VALUES (1850);
```

Output:

```
ERROR:  value too long for type character varying(10)
ERROR:  smallint out of range
ERROR:  numeric field overflow
DETAIL:  A field with precision 10, scale 2 must round to an absolute value less than 10^8.
ERROR:  date/time field value out of range: "2025-02-30"
ERROR:  new row for relation "type_demo" violates check constraint "type_demo_birth_year_check"
...
```

`numeric(10,2)` allows 10 digits of which 2 are after the point, so at most 8 before it ($< 10^8$). MySQL in non-strict mode (a lenient setting that turns many errors into warnings) would silently truncate some of these values instead of failing; PostgreSQL always errors.

How to choose

1. **Money** → `numeric(p, s)` (or integer cents), never `float`. **Ids/counts** → `integer`, or `bigint` if it could ever pass 2 billion.
2. **Text** → `text` in PostgreSQL; `varchar(n)` only when the maximum is a real business rule (no speed difference). SQL Server: `nvarchar` for Unicode.
3. **Instants** → `timestamptz`; `timestamp` only for wall-clock times that must not shift (a shop's opening time). **Days** → `date`.
4. Pick the smallest type that safely fits — changing it later on a big table is expensive (9.5).

Common mistakes

Money in `float` (rounding errors); dates as `text` (`'03/04/2025'` — March or April? can't sort); `varchar(255)` everywhere by habit (a meaningless limit); `timestamp` for events across time zones (ambiguous instants).

9.2 CREATE: databases, schemas and tables

What it is

`CREATE` makes new database objects. The ones you'll use most:

Statement	Creates
`CREATE DATABASE name;`	a new, separate database on the server (the running database program)
`CREATE SCHEMA name;`	a namespace **inside** a database (like a folder); tables are then `schema.table`
`CREATE TABLE name (...);`	a table: columns, types, constraints
`CREATE TABLE name AS SELECT ...;`	a table filled with a query's result
`CREATE TABLE name (LIKE other INCLUDING ALL);`	an empty copy of another table's structure
`CREATE TEMP TABLE name (...);`	a table visible only to your session, dropped automatically at the end
`CREATE VIEW`, `CREATE INDEX`, `CREATE SEQUENCE`, `CREATE FUNCTION`…	covered in chapters 7, 9.7, 9.4, 14

How CREATE TABLE works

```
CREATE TABLE [IF NOT EXISTS] [schema.]table_name (
    column_name  data_type  [column constraints],   -- one line per column
    ...,
    [table constraints]                             -- e.g. PRIMARY KEY (a, b)
);
```

Square brackets in a syntax outline like this mark optional parts — you don't type the brackets.

1. The name must be unique within its schema (default `public`); each column gets a name, a type and optionally constraints (9.3) and a `DEFAULT`.
2. Multi-column constraints go after the columns.
3. `IF NOT EXISTS` turns "already exists" into a notice (an informational message, not an error) — handy in setup scripts, but it does **not** check that the existing table has the same columns.

```sql
-- code/02_create.sql
CREATE DATABASE school_demo;          -- a new database (then dropped again in the script)
CREATE SCHEMA school;

CREATE TABLE school.students (
    student_id   integer      PRIMARY KEY,
    first_name   varchar(50)  NOT NULL,
    last_name    varchar(50)  NOT NULL,
    roll_number  integer      UNIQUE,
    email        text,
    enrolled_on  date         NOT NULL DEFAULT current_date
);

CREATE TABLE IF NOT EXISTS school.students (student_id integer);

CREATE TABLE school.bad (
    id   integer PRIMARY KEY,
    name text,
);
```

Output:

```
NOTICE:  relation "students" already exists, skipping
ERROR:  syntax error at or near ")"
LINE 4: );
```

A comma after the last column is a syntax error (PostgreSQL, MySQL, SQL Server) — a common typo after deleting the last line.

Tables from queries and copies:

```sql
CREATE TABLE school.sales_staff AS
SELECT emp_id, first_name, salary FROM employees WHERE dept_id = 2;
SELECT * FROM school.sales_staff ORDER BY emp_id;
```

Output:

```
 emp_id | first_name |  salary
--------+------------+----------
      6 | Daniel     | 150000.00
      7 | Priya      |  72000.00
 ...
(4 rows)
```

`CREATE TABLE … AS` copies column names, types and data, but **no constraints, defaults or indexes**. (`SELECT … INTO new_table FROM …` does the same in PostgreSQL and SQL Server.) To copy only the structure, with constraints and defaults, use `LIKE`:

```sql
CREATE TABLE school.products_copy (LIKE products INCLUDING ALL);   -- 0 rows, same columns/constraints
```

Temporary tables (`CREATE TEMP TABLE scratch (id int, note text);`) live in a private per-session schema (the file's catalog query — the **catalog** is PostgreSQL's set of internal tables describing every object — shows a `pg_temp_N` schema and `relpersistence = 't'` instead of `'p'`) and vanish when the session ends — or at transaction end with `ON COMMIT DROP`. Use them for multi-step processing inside one session (7.9).

> **Key term — transaction:** a group of statements the database treats as one unit: either all of them take effect or none do. You start one with `BEGIN`, make it permanent with `COMMIT`, or undo everything since `BEGIN` with `ROLLBACK`. Without `BEGIN`, PostgreSQL wraps each statement in its own tiny transaction. Transactions are covered in depth in chapter 13.

Dialect notes

	PostgreSQL	MySQL	SQL Server	SQLite
Database vs schema	database contains schemas	"schema" = "database" (synonyms)	database contains schemas (`dbo` default)	one file = one database
Switch database	reconnect (`\c db` in psql, PostgreSQL's command-line client)	`USE db;`	`USE db;`	open another file / `ATTACH`
`CREATE TABLE IF NOT EXISTS`	yes	yes	no (check `OBJECT_ID` first)	yes
Temporary table	`CREATE TEMP TABLE`	`CREATE TEMPORARY TABLE`	`CREATE TABLE #name`	`CREATE TEMP TABLE`
`CREATE DATABASE` inside a transaction	not allowed	implicit commit (silently ends the open transaction first)	not allowed	n/a

---

9.3 Keys and constraints

What they are

A **key** is a column (or set of columns) whose values identify rows. A **constraint** is a rule the database checks on every write; a statement that would break it fails completely. Applications have bugs and there is usually more than one of them, but every write passes through the database — so that is where the rules belong.

The kinds of keys

Think of a `members` table with `member_id`, `email`, `phone`, `full_name`:

Key	Definition	In `members`
Super key	any column set that is unique for every row (may include unnecessary columns)	`{member_id}`, `{member_id, full_name}`, `{email, phone}`, …
Candidate key	a *minimal* super key — drop any column and it stops being unique; a table can have several	`{member_id}`, `{email}`
Primary key (PK)	the candidate key chosen as main identifier: unique **and** `NOT NULL`, one per table	`member_id`
Alternate key	a candidate key that wasn't chosen as primary; enforced with `UNIQUE`	`email`
Unique key	columns declared `UNIQUE`: no duplicates, NULLs allowed (below); many per table	`email`, `phone`
Composite key	two or more columns; only the *combination* must be unique	`(club_id, member_id)` in `club_memberships`
Foreign key (FK)	column(s) whose values must match a key (usually the PK) of another (or the same) table	`club_memberships.member_id → members.member_id`
Natural vs surrogate	natural = a real-world value (email, ISBN); surrogate = a generated number/UUID (9.4)	`email` is natural, `member_id` is surrogate

Keys and their theory (functional dependencies) are developed further in chapter 10 and chapter 11.

The constraints

Constraint	Rule	Written as
`NOT NULL`	a value is required	`col type NOT NULL`
`DEFAULT`	value used when an `INSERT` omits the column (not a check, but declared alongside)	`col type DEFAULT expr`
`UNIQUE`	no two rows share the value(s); NULLs don't count as equal	`col type UNIQUE` / `UNIQUE (a, b)`
`PRIMARY KEY`	`UNIQUE` + `NOT NULL`; one per table; creates a unique index (a lookup structure, 9.7)	`col type PRIMARY KEY` / `PRIMARY KEY (a, b)`
`CHECK`	a boolean expression must not be false	`CHECK (price >= 0)` / `CHECK (end_date >= start_date)`
`FOREIGN KEY`	value must exist in the referenced key (or be NULL)	`col type REFERENCES other(col)` / `FOREIGN KEY (a) REFERENCES other(x)`
`EXCLUDE` (PostgreSQL)	no two rows may "conflict" by an operator (e.g. overlapping bookings of one room)	see chapter 14

A **column constraint** is written on the column; a **table constraint** after the columns (required for multi-column rules). Name important ones with `CONSTRAINT name …` — the name appears in errors and is needed to drop it; otherwise PostgreSQL generates names like `members_email_key`.

```
-- code/03_keys_and_constraints.sql
CREATE TABLE members (
    member_id  integer PRIMARY KEY,                       -- primary key
    email      text    NOT NULL UNIQUE,                   -- candidate key -> alternate key
    phone      text    UNIQUE,                            -- unique key, NULL allowed
    full_name  text    NOT NULL,
    plan       text    NOT NULL DEFAULT 'free'
                       CONSTRAINT plan_is_known CHECK (plan IN ('free', 'pro', 'team')),
    joined_on  date    NOT NULL DEFAULT current_date,
    left_on    date,
    CONSTRAINT left_after_joined CHECK (left_on IS NULL OR left_on >= joined_on)   -- table-level check
);

INSERT INTO members (member_id, email, phone, full_name, joined_on)
VALUES (1, 'ravi@mail.example', NULL, 'Ravi Kumar', '2025-01-10'),
       (2, 'lena@mail.example', NULL, 'Lena Ortiz', '2025-02-01');     -- two NULL phones: fine
SELECT member_id, email, phone, plan FROM members ORDER BY member_id;
```

Output:

```
 member_id |       email       | phone  | plan
-----------+-------------------+--------+------
         1 | ravi@mail.example | (null) | free
         2 | lena@mail.example | (null) | free
(2 rows)
```

`plan` was filled from its `DEFAULT`. Now each rule refusing bad data:

```
INSERT INTO members (member_id, email, full_name) VALUES (1, 'x@mail.example', 'Dup Id');
INSERT INTO members (member_id, email, full_name, joined_on) VALUES (NULL, 'y@mail.example', 'No Id', '2025-03-01');
INSERT INTO members (member_id, email, full_name) VALUES (3, 'ravi@mail.example', 'Dup');
INSERT INTO members (member_id, email, full_name, plan, joined_on) VALUES (4, 'z@mail.example', 'Z', 'gold', '2025-03-01');
INSERT INTO members (member_id, email, full_name, joined_on, left_on)
VALUES (5, 'w@mail.example', 'W', '2025-03-01', '2025-01-01');
```

Output:

```
ERROR:  duplicate key value violates unique constraint "members_pkey"
DETAIL:  Key (member_id)=(1) already exists.
ERROR:  null value in column "member_id" of relation "members" violates not-null constraint
...
ERROR:  duplicate key value violates unique constraint "members_email_key"
...
ERROR:  new row for relation "members" violates check constraint "plan_is_known"
...
ERROR:  new row for relation "members" violates check constraint "left_after_joined"
...
```

Two subtle rules:

- **`CHECK` passes when its expression is `NULL`.** A check fails only when the expression is *false*; `NULL` ("unknown") is accepted. That's why `left_after_joined` explicitly allows `left_on IS NULL`, and why a column that must be present also needs `NOT NULL`. (Three-valued logic — every condition is true, false or unknown: chapter 2.)
- **`UNIQUE` allows several `NULL`s** in PostgreSQL, MySQL, SQLite and Oracle, because NULL is not equal to NULL. SQL Server allows only **one** NULL in a unique constraint (use a filtered unique index — an index over only the rows matching a `WHERE` — for "unique when present"). PostgreSQL 15 added `UNIQUE NULLS NOT DISTINCT` to treat NULLs as equal (PostgreSQL 15 — not run).

Foreign keys and referential actions

A foreign key keeps **referential integrity**: a child row can't point to a parent that doesn't exist. The interesting question is what happens when the *parent* is deleted or its key changes. You choose with `ON DELETE` / `ON UPDATE`:

Action	When the referenced parent row is deleted (or its key updated)
`NO ACTION` (default)	error if children still reference it (checked at the end of the statement; can be deferred)
`RESTRICT`	error, checked immediately (can't be deferred)
`CASCADE`	delete (or update) the child rows too
`SET NULL`	set the child's FK column(s) to NULL
`SET DEFAULT`	set the child's FK column(s) to their default

```
CREATE TABLE clubs (
    club_id  integer PRIMARY KEY,
    name     text NOT NULL UNIQUE
);
CREATE TABLE club_memberships (
    club_id    integer REFERENCES clubs(club_id)     ON DELETE CASCADE,
    member_id  integer REFERENCES members(member_id) ON DELETE RESTRICT,
    role       text NOT NULL DEFAULT 'member',
    PRIMARY KEY (club_id, member_id)                                  -- composite key
);
CREATE TABLE club_events (
    event_id   integer PRIMARY KEY,
    club_id    integer REFERENCES clubs ON DELETE SET NULL,           -- "REFERENCES clubs" = its PK
    title      text NOT NULL
);
INSERT INTO clubs VALUES (10, 'Chess'), (20, 'Hiking');
INSERT INTO club_memberships (club_id, member_id) VALUES (10, 1), (10, 2), (20, 1);
INSERT INTO club_events VALUES (100, 10, 'Blitz night'), (101, 20, 'Sunrise trek');

INSERT INTO club_memberships (club_id, member_id) VALUES (10, 1);       -- composite PK duplicate
INSERT INTO club_memberships (club_id, member_id) VALUES (30, 1);       -- no club 30
DELETE FROM members WHERE member_id = 2;                                  -- RESTRICT
```

Output:

```
ERROR:  duplicate key value violates unique constraint "club_memberships_pkey"
...
ERROR:  insert or update on table "club_memberships" violates foreign key constraint
"club_memberships_club_id_fkey"
DETAIL:  Key (club_id)=(30) is not present in table "clubs".
ERROR:  update or delete on table "members" violates foreign key constraint "club_memberships_member_id_fkey" on
table "club_memberships"
DETAIL:  Key (member_id)=(2) is still referenced from table "club_memberships".
```

Deleting the Chess club triggers both `CASCADE` (its memberships go) and `SET NULL` (its event stays, orphaned on purpose):

```
DELETE FROM clubs WHERE club_id = 10;
SELECT * FROM club_memberships ORDER BY club_id, member_id;
SELECT * FROM club_events ORDER BY event_id;
```

Output:

```
 club_id | member_id |  role
---------+-----------+--------
      20 |         1 | member
(1 row)

 event_id | club_id |    title
----------+---------+--------------
      100 |  (null) | Blitz night
      101 |      20 | Sunrise trek
(2 rows)
```

Choosing an action: `CASCADE` for rows that are *part of* the parent (order lines, memberships); `RESTRICT` / `NO ACTION` for things that must not vanish silently (a customer with orders); `SET NULL` when the child should survive without the link.

Every constraint is recorded in the `pg_constraint` catalog (`contype` `p` = primary key, `u` = unique, `f` = foreign key, `c` = check); the file lists the eight created here.

Deferrable constraints. Constraints are normally checked after each statement; `DEFERRABLE INITIALLY DEFERRED` postpones the check to `COMMIT`, allowing temporarily inconsistent states — e.g. two rows that reference each other:

```
CREATE TABLE people (
    person_id  integer PRIMARY KEY,
    partner_id integer REFERENCES people DEFERRABLE INITIALLY DEFERRED
);
BEGIN;
INSERT INTO people VALUES (1, 2);   -- person 2 doesn't exist yet: allowed for now
INSERT INTO people VALUES (2, 1);
COMMIT;                             -- checked here: both exist
```

Both rows are committed. (`UNIQUE`, `PRIMARY KEY`, `REFERENCES` and `EXCLUDE` can be deferrable in PostgreSQL; `CHECK` and `NOT NULL` cannot.)

Dialect notes

InnoDB is MySQL's default **storage engine** (the component that actually stores and reads rows); SQLite's `PRAGMA` is its command for changing settings.

	PostgreSQL	MySQL (InnoDB)	SQL Server	SQLite
`CHECK` enforced	yes	since 8.0.16 (parsed but ignored before)	yes	yes
FK enforced	yes	yes	yes	only after `PRAGMA foreign_keys = ON`
Index auto-created on FK column	**no** — create one yourself	yes	no	no
Multiple NULLs in `UNIQUE`	yes	yes	no (one)	yes
Deferrable constraints	yes	no	no	FKs only

When not to rely on constraints alone

Constraints check single rows (and FK existence). Multi-row rules ("at most 5 members per club") need triggers (functions the database runs automatically when rows are inserted, updated or deleted), `EXCLUDE` constraints or careful transactions (chapter 13, chapter 14).

Common mistakes

- **No index on the FK column**: in PostgreSQL each parent delete then scans the whole child table.
- **`ON DELETE CASCADE` everywhere**: one delete can wipe out large parts of the database.
- **Forgetting `NOT NULL`** next to `CHECK` or `UNIQUE` when a value is required.
- **A changeable natural key** (email, phone) as the PK — every FK must change with it. Use a surrogate PK plus `UNIQUE` on the natural key.

In the interview

30-second answer: "A candidate key is a minimal set of columns that identifies a row; the primary key is the chosen one — unique, not null, one per table; the rest are alternate keys enforced with `UNIQUE` (NULLs allowed). A composite key combines columns. A foreign key references another table's key, and `ON DELETE CASCADE / SET NULL / RESTRICT` decides what happens to children. `CHECK`, `NOT NULL` and `DEFAULT` complete the set."

- *Primary key vs unique key?* → PK: one per table, never NULL, the row's identity (often the clustered index — the one that stores the rows themselves in key order, 9.7 — in SQL Server/MySQL). UNIQUE: many per table, NULLs allowed.
- *Natural or surrogate PK?* → Usually a surrogate (`bigint` identity or UUID) for stability, plus `UNIQUE` on the natural key to keep the business rule.

Self-check

1. *Can a table have two primary keys?* → No; one primary key (possibly composite), any number of unique keys.
2. *Does `CHECK (qty > 0)` reject NULL?* → No; add `NOT NULL`.
3. *Does PostgreSQL index a foreign-key column automatically?* → No.
4. *Is `{member_id, email}` a candidate key?* → No — it's a super key but not minimal (`member_id` alone is unique).

9.4 Generated keys: identity, sequences, serial, UUID, generated columns

What it is

Most tables use a generated **surrogate key**. PostgreSQL generates numbers with a **sequence** — an object that hands out increasing values (`nextval`); an **identity column** (SQL standard, PostgreSQL 10+) is a column tied to its own sequence.

Syntax	Meaning
`GENERATED ALWAYS AS IDENTITY`	database always generates the value; supplying one is an error (unless `OVERRIDING SYSTEM VALUE`)
`GENERATED BY DEFAULT AS IDENTITY`	generated when you don't supply one; you may supply your own
`serial` / `bigserial`	older PostgreSQL shorthand: integer column + sequence + `DEFAULT nextval(...)`
`uuid DEFAULT gen_random_uuid()`	random 128-bit id — a UUID, *universally unique identifier* (built in since PostgreSQL 13)
`GENERATED ALWAYS AS (expr) STORED`	a **generated column**: computed from other columns on every write (PostgreSQL 12+)

```
-- code/04_identity_and_sequences.sql
CREATE TABLE tickets (
    ticket_id  bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    subject    text NOT NULL,
    price      numeric(8,2) NOT NULL,
    qty        integer NOT NULL DEFAULT 1,
    total      numeric(10,2) GENERATED ALWAYS AS (price * qty) STORED
);
INSERT INTO tickets (subject, price) VALUES ('Printer jam', 10);
INSERT INTO tickets (subject, price, qty) VALUES ('VPN access', 5, 3), ('New laptop', 1200, 1)
RETURNING ticket_id, total;
```

Output:

```
 ticket_id |  total
-----------+---------
         2 |   15.00
         3 | 1200.00
(2 rows)
```

```
INSERT INTO tickets (ticket_id, subject, price) VALUES (99, 'Manual id', 1);
INSERT INTO tickets (subject, price, total) VALUES ('Fake total', 1, 500);
```

Output:

```
ERROR:  cannot insert a non-DEFAULT value into column "ticket_id"
DETAIL:  Column "ticket_id" is an identity column defined as GENERATED ALWAYS.
HINT:  Use OVERRIDING SYSTEM VALUE to override.
ERROR:  cannot insert a non-DEFAULT value into column "total"
DETAIL:  Column "total" is a generated column.
```

Sequences never go backwards — gaps are normal:

```
INSERT INTO tickets (ticket_id, subject, price) OVERRIDING SYSTEM VALUE VALUES (99, 'Manual id', 1);
BEGIN;
INSERT INTO tickets (subject, price) VALUES ('Will be rolled back', 1);
ROLLBACK;
INSERT INTO tickets (subject, price) VALUES ('After rollback', 2);
SELECT ticket_id, subject, total FROM tickets ORDER BY ticket_id;
```

Output:

```
 ticket_id |    subject     |  total
-----------+----------------+---------
         1 | Printer jam    |   10.00
 ...
         3 | New laptop     | 1200.00
         5 | After rollback |    2.00
        99 | Manual id      |    1.00
(5 rows)
```

Number 4 went to the rolled-back insert and is never reused: sequences are deliberately **not** transactional (a rollback doesn't undo them), so concurrent inserts (inserts running at the same moment in different sessions) never wait for each other. Never assume ids are gap-free (for legal invoice numbers, generate them separately under a lock — a **lock** is a claim on a row or table that makes other sessions wait until you finish).

Why identity beats `serial`: `serial` is just a default, so a manual id doesn't advance the sequence and a later generated one collides:

```
CREATE TABLE legacy (id serial PRIMARY KEY, label text);
INSERT INTO legacy (label) VALUES ('a'), ('b');       -- ids 1, 2
INSERT INTO legacy (id, label) VALUES (3, 'manual');  -- allowed
INSERT INTO legacy (label) VALUES ('c');              -- sequence gives 3 -> collision
```

Output:

```
ERROR:  duplicate key value violates unique constraint "legacy_pkey"
DETAIL:  Key (id)=(3) already exists.
```

`GENERATED ALWAYS` refuses the manual insert up front. (After loading your own ids, resync: `SELECT setval('legacy_id_seq', (SELECT max(id) FROM legacy));`.)

A standalone sequence, e.g. for invoice numbers shared by several tables:

```
CREATE SEQUENCE invoice_no START 1000 INCREMENT 10;
SELECT nextval('invoice_no') AS first, nextval('invoice_no') AS second, currval('invoice_no') AS current;
-- 1000 | 1010 | 1010
```

`currval` returns the last value *this session* got from `nextval`.

UUIDs: `key_id uuid PRIMARY KEY DEFAULT gen_random_uuid()` gives 128-bit ids (36 characters as text). They can be generated anywhere (even in the app) without coordination and don't reveal row counts, but take 16 bytes (vs 8 for `bigint`) and random values scatter index inserts. Time-ordered UUIDv7 (a UUID whose first bits are a timestamp, so new ids sort roughly by creation time) reduces that; PostgreSQL 18 adds a built-in `uuidv7()` (not run here).

Dialect notes

	Auto-increment syntax
PostgreSQL	`GENERATED {ALWAYS | BY DEFAULT} AS IDENTITY` (10+), `serial` (legacy)
MySQL	`AUTO_INCREMENT`
SQL Server	`IDENTITY(1,1)`; `SEQUENCE` objects since 2012
SQLite	`INTEGER PRIMARY KEY` (alias of the rowid, SQLite's hidden row number); `AUTOINCREMENT` prevents reuse of deleted ids
Oracle	identity columns since 12c; sequences

Generated columns: PostgreSQL 12–17 `STORED` only; PostgreSQL 18 adds `VIRTUAL` (computed on read) and makes it the default when neither keyword is given (not run); MySQL 5.7+ supports both; SQL Server has "computed columns" (optionally `PERSISTED`).

9.5 ALTER: changing existing tables

What it is

`ALTER TABLE` changes an existing table's structure — usually with data in it — without recreating it.

Task	PostgreSQL
Add a column	`ALTER TABLE t ADD COLUMN c type [DEFAULT …] [NOT NULL];`
Drop a column	`ALTER TABLE t DROP COLUMN c;`
Rename a column / table	`ALTER TABLE t RENAME COLUMN a TO b;` / `ALTER TABLE t RENAME TO u;`
Change a type	`ALTER TABLE t ALTER COLUMN c TYPE newtype [USING expr];`
Set / drop a default	`ALTER TABLE t ALTER COLUMN c SET DEFAULT …;` / `DROP DEFAULT`
Set / drop NOT NULL	`ALTER TABLE t ALTER COLUMN c SET NOT NULL;` / `DROP NOT NULL`
Add / drop a constraint	`ALTER TABLE t ADD CONSTRAINT name …;` / `DROP CONSTRAINT name;`

How it works (with real data)

```sql
-- code/05_alter.sql
CREATE TABLE staff (id integer PRIMARY KEY, first_name text, last_name text);
INSERT INTO staff VALUES (1, 'Tara', 'Mendes'), (2, 'Omar', 'Mendes'), (3, 'Yuki', 'Tanaka');

ALTER TABLE staff ADD COLUMN salary integer;
ALTER TABLE staff ADD COLUMN country text NOT NULL DEFAULT 'US';
SELECT * FROM staff ORDER BY id;
```

Output:

```
 id | first_name | last_name | salary | country
----+------------+-----------+--------+---------
  1 | Tara       | Mendes    | (null) | US
  2 | Omar       | Mendes    | (null) | US
  3 | Yuki       | Tanaka    | (null) | US
(3 rows)
```

Existing rows get NULL, or the default if there is one. Since PostgreSQL 11, adding a column with a constant default is instant (no table rewrite — copying every row into a new file).

```sql
ALTER TABLE staff RENAME COLUMN first_name TO given_name;
ALTER TABLE staff RENAME TO team;

UPDATE team SET salary = 50000 + id * 1000;
ALTER TABLE team ALTER COLUMN salary TYPE numeric(10,2) USING salary::numeric(10,2);
ALTER TABLE team ALTER COLUMN last_name TYPE integer;                -- text -> integer?
```

Output:

```
ERROR:  column "last_name" cannot be cast automatically to type integer
HINT:  You might need to specify "USING last_name::integer".
```

`USING` tells PostgreSQL how to compute the new value from the old one; without it only automatic (implicit/assignment) casts — the conversions PostgreSQL considers safe to apply on its own — are allowed.

Adding `NOT NULL` to a column that has NULLs fails — `ALTER TABLE team ADD COLUMN email text NOT NULL;` gives *ERROR: column "email" of relation "team" contains null values*. Add, backfill (fill in values for the existing rows), then constrain:

```
ALTER TABLE team ADD COLUMN email text;
UPDATE team SET email = lower(given_name) || '@corp.example';
ALTER TABLE team ALTER COLUMN email SET NOT NULL;
```

Constraints on live tables: `NOT VALID` then `VALIDATE`. Adding a `CHECK` or FK normally scans the whole table under a strong lock. On a big table, add it `NOT VALID` (enforced for new and changed rows only), fix old data, then `VALIDATE CONSTRAINT` (scans under a lighter lock):

```
ALTER TABLE team ADD CONSTRAINT team_email_uq UNIQUE (email);
INSERT INTO team (id, given_name, last_name, salary, email) VALUES (4, 'Leo', 'Brandt', -5, 'leo@corp.example');
ALTER TABLE team ADD CONSTRAINT salary_positive CHECK (salary > 0);           -- fails: Leo's row
ALTER TABLE team ADD CONSTRAINT salary_positive CHECK (salary > 0) NOT VALID;  -- accepted
ALTER TABLE team VALIDATE CONSTRAINT salary_positive;                         -- fails until fixed
UPDATE team SET salary = 48000 WHERE id = 4;
ALTER TABLE team VALIDATE CONSTRAINT salary_positive;                         -- succeeds
```

Output:

```
ERROR:  check constraint "salary_positive" of relation "team" is violated by some row
ERROR:  check constraint "salary_positive" of relation "team" is violated by some row
```

(`UNIQUE`/`PRIMARY KEY` can't be `NOT VALID`; build a unique index `CONCURRENTLY` (without blocking writes, 9.7) and then `ADD CONSTRAINT … UNIQUE USING INDEX` instead.)

Several changes in one statement (one lock, one pass):

```
ALTER TABLE team
    DROP COLUMN country,
    ADD COLUMN hired_on date,
    ALTER COLUMN last_name SET NOT NULL;
SELECT conname, pg_get_constraintdef(oid) AS definition
FROM pg_constraint WHERE conrelid = 'team'::regclass ORDER BY conname;
```

Output:

```
     conname     |           definition
-----------------+--------------------------------
 salary_positive | CHECK ((salary > (0)::numeric))
 staff_pkey      | PRIMARY KEY (id)
 team_email_uq   | UNIQUE (email)
(3 rows)
```

Note the primary key is still called `staff_pkey`: renaming a table doesn't rename its constraints or indexes.

Locks and production safety

Most `ALTER TABLE` forms take an **ACCESS EXCLUSIVE** lock — no reads or writes until it finishes. Metadata-only changes (edits to the table's description in the catalog that don't touch its rows: nullable column, constant default, rename, drop column — which only hides it) hold it briefly; **type changes that rewrite the table** can hold it for minutes. Production (the live system real users depend on) practice: set `lock_timeout` (so a statement gives up instead of waiting indefinitely for a lock), use `NOT VALID` + `VALIDATE` and `CREATE INDEX CONCURRENTLY`, and "expand → migrate → contract" (add the new column, backfill in batches — chunks of a few thousand rows at a time — switch the app, drop the old one). Locks: chapter 13.

Dialect notes

Task	PostgreSQL	MySQL	SQL Server
Change type	`ALTER COLUMN c TYPE t`	`MODIFY COLUMN c t`	`ALTER COLUMN c t`
Rename column	`RENAME COLUMN a TO b`	`RENAME COLUMN a TO b` (8.0) / `CHANGE a b t`	`EXEC sp_rename 't.a', 'b', 'COLUMN'`
Rename table	`ALTER TABLE t RENAME TO u`	`RENAME TABLE t TO u`	`EXEC sp_rename 't', 'u'`
DDL inside a transaction (rollback-able)	yes	no — implicit commit	yes (mostly)

> **Common mistakes**

`NOT NULL` without a default or backfill (fails); a type change on a huge table in business hours (long exclusive lock); renaming columns that application SQL still uses (views follow renames automatically in PostgreSQL, application code does not).

9.6 DROP and TRUNCATE (vs DELETE)

What they are

- `DROP` removes an object entirely: the table's definition, data, indexes, constraints, triggers and permissions. `DROP TABLE t;`, `DROP DATABASE d;`, `DROP SCHEMA s;`, `DROP VIEW v;`, `DROP INDEX i;`.
- `TRUNCATE` removes **all rows** from a table but keeps the table itself (structure, indexes, grants). `TRUNCATE TABLE t;` (the word `TABLE` is optional in PostgreSQL and MySQL, required in SQL Server and Oracle).
- `DELETE` (DML, 9.10) removes rows one by one, optionally only those matching a `WHERE`.

Analogy: `DELETE` erases lines from a notebook one by one; `TRUNCATE` swaps in a fresh empty notebook; `DROP` throws the notebook away.

How TRUNCATE works

```sql
-- code/06_drop_and_truncate.sql
CREATE TABLE audit_log (log_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY, msg text NOT NULL);
INSERT INTO audit_log (msg) SELECT 'event ' || g FROM generate_series(1, 5) g;

TRUNCATE audit_log;
SELECT count(*) AS rows_after_truncate FROM audit_log;                    -- 0
INSERT INTO audit_log (msg) VALUES ('after truncate') RETURNING log_id;   -- 6: identity continues
TRUNCATE audit_log RESTART IDENTITY;
INSERT INTO audit_log (msg) VALUES ('after restart') RETURNING log_id;    -- 1
```

Output (the three results): `0`, then `log_id = 6`, then `log_id = 1`.

In PostgreSQL, `TRUNCATE` is **transactional** — you can roll it back:

```sql
BEGIN;
TRUNCATE audit_log;
SELECT count(*) AS inside_tx FROM audit_log;       -- 0
ROLLBACK;
SELECT count(*) AS after_rollback FROM audit_log;  -- 1
```

A table referenced by a foreign key can't be truncated alone, because that would orphan the child rows; `CASCADE` truncates the referencing tables too:

```
TRUNCATE customers;
BEGIN;
TRUNCATE customers CASCADE;
SELECT (SELECT count(*) FROM customers) AS customers,
       (SELECT count(*) FROM orders) AS orders,
       (SELECT count(*) FROM order_items) AS items;
ROLLBACK;
```

Output:

```
ERROR:  cannot truncate a table referenced in a foreign key constraint
DETAIL:  Table "orders" references "customers".
HINT:  Truncate table "orders" at the same time, or use TRUNCATE ... CASCADE.
NOTICE:  truncate cascades to table "orders"
NOTICE:  truncate cascades to table "order_items"
 customers | orders | items
-----------+--------+-------
         0 |      0 |     0
(1 row)
```

How DROP works

`DROP TABLE audit_log;` removes the table; querying it afterwards gives *relation "audit_log" does not exist*, a second `DROP` errors, and `DROP TABLE IF EXISTS` only prints a notice — the form to use in scripts. `DROP` refuses while other objects depend on the table (default `RESTRICT`); `CASCADE` drops the dependents too — views are dropped, and FKs *pointing at* the table are removed (the child table and its rows stay):

```
CREATE VIEW big_spenders AS SELECT customer_id FROM orders GROUP BY customer_id HAVING count(*) > 2;
DROP TABLE orders;
BEGIN;
DROP TABLE orders CASCADE;
SELECT count(*) AS order_items_still_there FROM order_items;
ROLLBACK;
```

Output:

```
ERROR:  cannot drop table orders because other objects depend on it
DETAIL:  constraint order_items_order_id_fkey on table order_items depends on table orders
view big_spenders depends on table orders
HINT:  Use DROP ... CASCADE to drop the dependent objects too.
NOTICE:  drop cascades to 2 other objects
...
 order_items_still_there
-------------------------
                      22
(1 row)
```

`DROP DATABASE name;` removes a whole database (you can't be connected to it, and in PostgreSQL it can't run inside a transaction). `DROP SCHEMA s CASCADE;` removes a schema and everything in it.

DELETE vs TRUNCATE vs DROP

	`DELETE FROM t [WHERE …]`	`TRUNCATE t`	`DROP TABLE t`
Kind	DML	DDL	DDL
Removes	chosen rows (or all)	all rows	rows **and** the table
`WHERE` possible	yes	no	no
Speed on big tables	slow: row by row, all logged (each change written to the transaction log)	fast: releases the data files	fast
Space returned to the OS	usually not: dead rows (old, deleted versions) are reused after `VACUUM` (PostgreSQL's cleanup process), the file rarely shrinks	yes	yes
Row-level `ON DELETE` triggers	fire	don't fire (PostgreSQL has `ON TRUNCATE` triggers)	n/a
FK checks	per row (or cascades)	refused if referenced, unless `CASCADE`	refused if referenced, unless `CASCADE`
Identity / auto-increment	continues	PostgreSQL: continues unless `RESTART IDENTITY`; MySQL & SQL Server: reset	gone
Rollback	yes	inside a transaction in PostgreSQL & SQL Server; **not** in MySQL & Oracle (implicit commit)	same as `TRUNCATE`
Lock	row locks	exclusive table lock	exclusive table lock

"Can `TRUNCATE`/`DROP` be undone?" therefore depends on the database — in MySQL the only way back is a backup (9.12); Oracle can undo a `DROP TABLE` from its recycle bin (`FLASHBACK TABLE t TO BEFORE DROP`, if enabled) but not a `TRUNCATE`. SQLite has no `TRUNCATE`; `DELETE FROM t` without `WHERE` is optimized to do the same job.

Common mistakes

- Running `DROP`/`TRUNCATE` on the wrong connection (production instead of dev) — make production visually distinct and use reviewed migrations (versioned, checked-in scripts that change the schema step by step).
- `… CASCADE` without reading what it cascades to — try it inside `BEGIN … ROLLBACK` first, as above.
- `DELETE` to empty a 100-million-row table (hours, huge log) when `TRUNCATE` takes a second.
- Expecting `TRUNCATE` to be invisible to concurrent transactions: it isn't MVCC-safe (MVCC, *multi-version concurrency control*, lets each transaction read a **snapshot** — a frozen, consistent view of the data as of one moment; chapter 13) — a transaction with an older snapshot that reads the table afterwards sees it empty.

In the interview

> **30-second answer:** "`DELETE` is DML: selected rows, one at a time, fires triggers, keeps the table. `TRUNCATE` is DDL: all rows at once by deallocating storage — much faster, no `WHERE`, no delete triggers, often resets identity; rollback-able in PostgreSQL and SQL Server, not in MySQL/Oracle. `DROP` removes the table itself."

Self-check

1. *Empty a staging table (a scratch table where incoming data lands before processing) quickly and reset its ids in PostgreSQL?* → `TRUNCATE staging RESTART IDENTITY;`
2. *Can you `TRUNCATE` a table that other tables reference by FK?* → Only with `CASCADE` (or truncating them in the same statement).
3. *What does `DROP TABLE orders CASCADE` do to `order_items`?* → Drops the FK constraint on it; the table and rows remain.

9.7 Indexes (in brief)

An **index** is a separate data structure (by default a B-tree — a balanced, always-sorted tree of keys) that finds rows by column values without reading the whole table — like a book's index sending you straight to the page. This section shows only the DDL; how indexes work, their kinds, `EXPLAIN` (the command that shows how the database plans to run a query), and when they help or hurt are in chapter 12.

```sql
-- code/07_indexes_brief.sql
CREATE INDEX idx_employees_last_name ON employees (last_name);
CREATE INDEX idx_orders_customer_placed ON orders (customer_id, placed_at DESC);   -- composite

-- A UNIQUE index can enforce rules a UNIQUE constraint can't, e.g. case-insensitive uniqueness:
CREATE UNIQUE INDEX idx_customers_email_ci ON customers (lower(trim(email)));
```

Output:

```
ERROR:  could not create unique index "idx_customers_email_ci"
DETAIL:  Key (lower(TRIM(BOTH FROM email)))=(anita.shah@mail.example) is duplicated.
```

The sample data really has a duplicate customer (chapter 1's Anita Shah, entered twice with different capitalisation) — the index found it. After cleaning (`UPDATE customers SET email = NULL WHERE customer_id = 8`) the same statement succeeds.

Primary keys and unique constraints create their indexes automatically — `pg_indexes` (a catalog view listing every index) lists `employees_pkey` and `employees_email_key` next to the ones created above. Other index DDL: `DROP INDEX [IF EXISTS] name;`, and `CREATE INDEX CONCURRENTLY …` / `DROP INDEX CONCURRENTLY …`, which avoid blocking writes on a live table (slower, and not allowed inside a transaction block).

Clustered vs non-clustered

	Clustered index	**Non-clustered (secondary) index**
What it is	the table's rows are **stored in the index's order** — the index *is* the table	a separate structure of (key → pointer to the row)
How many	at most one per table	many
SQL Server	`CREATE CLUSTERED INDEX ix ON t (col);` (the PK is clustered by default)	`CREATE [NONCLUSTERED] INDEX ix ON t (col);`
MySQL InnoDB	always the primary key (no PK → the first `UNIQUE NOT NULL` index, else a hidden row id)	every other index; it stores the PK value as the pointer
PostgreSQL	**none** — tables are unordered "heaps"; `CLUSTER t USING ix;` re-sorts the table once, but new rows aren't kept in order	every index is secondary

```sql
CLUSTER orders USING idx_orders_customer_placed;   -- one-time physical re-ordering in PostgreSQL
```

A clustered index makes range scans (reading every key between two values) on its key fast (neighbouring keys are neighbouring rows) but makes the choice of key important: a random key (like a

random UUID) causes page splits on insert (a full storage page — the fixed-size block, 8 KB in PostgreSQL, that data is kept in — must be split in two to make room).

Cost of indexes: each index must be maintained on every insert and on updates of its columns (and cleaned up after deletes), and takes disk space. Index what queries filter, join and sort on — not every column.

9.8 INSERT

What it is

`INSERT` adds new rows to a table.

```
INSERT INTO table_name [(column1, column2, ...)]
VALUES (value1, value2, ...) [, (...), ...]
[ON CONFLICT ...]            -- 9.11
[RETURNING ...];             -- PostgreSQL: give back the inserted rows
-- or
INSERT INTO table_name [(columns)] SELECT ...;
```

How it works

1. The **column list** names the columns you supply, in order. Omitted columns get their `DEFAULT`, else NULL (an error if `NOT NULL` without a default).
2. Without a column list you must supply **every** column in table order — fragile: adding a column later breaks the statement.
3. Every row is checked against types and constraints; if any row fails, the **whole statement** fails.

```
-- code/08_insert.sql
CREATE TABLE contacts (
    contact_id  integer GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    first_name  text NOT NULL,
    last_name   text NOT NULL,
    age         integer CHECK (age >= 0),
    city        text DEFAULT 'Pune',
    created_at  timestamp NOT NULL DEFAULT '2025-06-01 09:00'
);

INSERT INTO contacts (first_name, last_name, age) VALUES ('Tara', 'Mendes', 28);

INSERT INTO contacts (first_name, last_name, age, city) VALUES      -- several rows at once
    ('Omar', 'Mendes', 31, 'Mumbai'),
    ('Yuki', 'Tanaka', NULL, DEFAULT);                              -- DEFAULT keyword

INSERT INTO contacts VALUES (DEFAULT, 'Leo', 'Brandt', 40, 'Delhi', '2025-06-02 10:00');  -- all columns

INSERT INTO contacts (first_name, last_name) VALUES ('Ines', 'Silva')
RETURNING contact_id, city, created_at;
```

Output (of the `RETURNING`):

```
 contact_id | city |     created_at
------------+------+---------------------
          5 | Pune | 2025-06-01 09:00:00
(1 row)
```

`RETURNING` hands back generated values (ids, defaults) in the same round trip (one request to the server and its reply) — no second query, no race (a **race condition** is a bug whose outcome depends on the timing of concurrent sessions).

`INSERT … SELECT` copies rows produced by any query:

```
INSERT INTO contacts (first_name, last_name, city)
SELECT first_name, last_name, city FROM customers WHERE country = 'UK'
RETURNING contact_id, first_name, city;
```

Output:

```
 contact_id | first_name |  city
------------+------------+--------
          6 | Ben        | London
          7 | George     | London
(2 rows)
```

Errors worth recognising:

```
INSERT INTO contacts (first_name, last_name) VALUES ("Ana", "Lima");
INSERT INTO contacts (first_name, last_name) VALUES ('Ana', 'Lima', 22);
INSERT INTO contacts (first_name, last_name, age) VALUES ('Ok', 'Row', 20), ('Bad', 'Row', -1);
SELECT count(*) AS still_7 FROM contacts;
```

Output:

```
ERROR:  column "Ana" does not exist
ERROR:  INSERT has more expressions than target columns
ERROR:  new row for relation "contacts" violates check constraint "contacts_age_check"
DETAIL:  Failing row contains (9, Bad, Row, -1, Pune, 2025-06-01 09:00:00).
 still_7
---------
       7
(1 row)
```

- In standard SQL (PostgreSQL, SQLite, SQL Server by default, Oracle) **double quotes name identifiers** (names of tables, columns and other objects); strings use single quotes. MySQL accepts double-quoted strings by default, so MySQL-style code often breaks elsewhere.
- The multi-row insert failed as a whole — the valid 'Ok' row was not kept (but it consumed identity values 8 and 9: more gaps).

Bulk loading: multi-row `VALUES` or `INSERT … SELECT` beats one statement per row; for millions of rows use `COPY table FROM file` (psql: `\copy` for a client file), MySQL `LOAD DATA INFILE`, SQL Server `BULK INSERT` / `bcp`.

Dialect notes

	PostgreSQL	MySQL	SQL Server	SQLite
Multi-row `VALUES`	yes	yes	yes (max 1000 rows per statement)	yes
Get generated values back	`RETURNING`	`LAST_INSERT_ID()`	`OUTPUT inserted.*` / `SCOPE_IDENTITY()`	`RETURNING` (3.35+) / `last_insert_rowid()`
`INSERT … DEFAULT VALUES`	yes	`INSERT INTO t () VALUES ()`	yes	yes

9.9 UPDATE

What it is

`UPDATE` changes column values in existing rows.

```
UPDATE table_name
SET column1 = expr1, column2 = expr2, ...
[FROM other_tables]          -- PostgreSQL: use other tables to compute / filter
[WHERE condition]            -- which rows; WITHOUT it, EVERY row changes
[RETURNING ...];
```

How it works

1. `WHERE` picks the rows, as in `SELECT`. No `WHERE` = all rows.
2. Every `SET` expression is evaluated from the row's **old** values, then all assignments apply together.
3. New rows must satisfy all constraints, or the statement fails and nothing changes.
4. PostgreSQL writes a new version of each updated row (MVCC — chapter 13); `VACUUM` later removes the old one.

Raise Sales salaries by 10 %:

```
-- code/09_update.sql
UPDATE employees
SET salary = salary * 1.10
WHERE dept_id = 2
RETURNING emp_id, first_name, salary;
```

Output:

```
 emp_id | first_name |  salary
--------+------------+-----------
      6 | Daniel     | 165000.00
      7 | Priya      |  79200.00
 ...
(4 rows)
```

Percentages are multiplications: +10 % is `* 1.10`, +50 % is `* 1.5`, doubling is `* 2` — check the factor against the requirement.

All `SET` expressions see the old row:

```
UPDATE products
SET unit_price   = unit_price * 0.9,
    discontinued = unit_price * 0.9 < 40          -- uses the OLD unit_price
WHERE category = 'Furniture'
RETURNING name, unit_price, discontinued;
```

Output:

```
      name       | unit_price | discontinued
-----------------+------------+--------------
 Standing Desk   |     449.10 | f
 Ergonomic Chair |     314.99 | f
 Desk Lamp       |      31.50 | t
(3 rows)
```

(In MySQL, assignments are applied left to right, so a later expression sees the *new* value of an earlier column — a real portability trap. PostgreSQL and SQL Server follow the standard.)

Different changes for different rows with `CASE`:

```
UPDATE orders
SET status = CASE WHEN shipped_at IS NULL THEN 'cancelled' ELSE 'delivered' END
WHERE status IN ('pending', 'shipped') AND placed_at < '2025-06-10'
RETURNING order_id, status;
```

Output:

```
 order_id |  status
----------+-----------
      107 | delivered
 ...
      113 | cancelled
(4 rows)
```

Using another table — PostgreSQL's `UPDATE … FROM` joins the target to other tables:

```
ALTER TABLE customers ADD COLUMN lifetime_orders integer NOT NULL DEFAULT 0;
UPDATE customers c
SET lifetime_orders = t.n
FROM (SELECT customer_id, count(*) AS n FROM orders GROUP BY customer_id) t
WHERE t.customer_id = c.customer_id;
SELECT customer_id, first_name, lifetime_orders FROM customers ORDER BY customer_id LIMIT 4;
```

Output:

```
 customer_id | first_name | lifetime_orders
-------------+------------+-----------------
           1 | Anita      |               3
           2 | Ben        |               2
 ...
(4 rows)
```

Make sure each target row matches **at most one** row of the `FROM` source; if it matches several, PostgreSQL uses one of them unpredictably. That's why the example aggregates first. The portable alternative is a correlated scalar subquery (a subquery that returns one value and is re-run for each outer row) in `SET`:

```
UPDATE departments d
SET budget = (SELECT coalesce(sum(salary), 0) * 1.5 FROM employees e WHERE e.dept_id = d.dept_id)
WHERE d.dept_id = 5
RETURNING name, budget;                          -- Support | 96000.00
```

The classic accident — and the habit that prevents it. `UPDATE` without `WHERE` updates every row. Practise risky statements inside a transaction, check the effect, then `COMMIT` or `ROLLBACK`:

```
BEGIN;
UPDATE employees SET salary = 1;
SELECT count(*) AS rows_now_paid_1 FROM employees WHERE salary = 1;
ROLLBACK;
SELECT count(*) AS rows_paid_1_after_rollback FROM employees WHERE salary = 1;
```

Output:

```
 rows_now_paid_1
-----------------
              15
(1 row)

 rows_paid_1_after_rollback
----------------------------
                          0
(1 row)
```

Other safety habits: run the `WHERE` as a `SELECT count(*)` first; check psql's `UPDATE n` row count; use `RETURNING`; in MySQL clients, `--safe-updates` refuses key-less updates.

Constraints still apply: setting `dept_id = 99` fails the foreign key, `unit_price = -1` fails the `CHECK` (both shown in the file).

9.10 DELETE

What it is

`DELETE` removes rows from a table.

```
DELETE FROM table_name
[USING other_tables]         -- PostgreSQL: join to other tables
[WHERE condition]            -- WITHOUT it, EVERY row is deleted
[RETURNING ...];
```

How it works

1. `WHERE` picks the rows; without it all rows go (the table remains).
2. Deleting a parent that still has children fails or cascades, as its foreign keys declare (9.3).
3. In PostgreSQL the rows are only marked dead; `VACUUM` reclaims the space later.

```
-- code/10_delete.sql
DELETE FROM order_items
WHERE order_id = 105
RETURNING order_id, product_id, quantity;
```

Output:

```
 order_id | product_id | quantity
----------+------------+----------
      105 |          1 |        2
(1 row)
```

```
DELETE FROM customers WHERE customer_id = 1;
```

Output:

```
ERROR:  update or delete on table "customers" violates foreign key constraint "orders_customer_id_fkey" on table
"orders"
DETAIL:  Key (customer_id)=(1) is still referenced from table "orders".
```

Deleting based on another table — PostgreSQL's `DELETE … USING`, or the portable subquery form:

```
DELETE FROM order_items oi
USING orders o
WHERE o.order_id = oi.order_id
  AND o.status = 'cancelled'
RETURNING oi.order_id, oi.product_id;          -- (111, 2)

DELETE FROM orders
WHERE status = 'cancelled'
  AND order_id NOT IN (SELECT order_id FROM order_items)
RETURNING order_id;                            -- 105, 111

DELETE FROM customers c
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id)
RETURNING customer_id, first_name;
```

Output of the last statement:

```
 customer_id | first_name
-------------+------------
           4 | Divya
           9 | Hana
          10 | Ivan
(3 rows)
```

(Divya's only order was a cancelled one, removed just before.) Children first, then parents — the order foreign keys demand. (`NOT IN` with a subquery is safe here only because `order_items.order_id` can't be NULL; prefer `NOT EXISTS` — see chapter 6.)

Delete everything — safely practised:

```
BEGIN;
DELETE FROM order_items;
SELECT count(*) AS items_inside_tx FROM order_items;       -- 0
ROLLBACK;
SELECT count(*) AS items_after_rollback FROM order_items;  -- 20
```

Soft delete. Many applications never physically delete business data; they mark it:

```
ALTER TABLE products ADD COLUMN deleted_at timestamp;
UPDATE products SET deleted_at = '2025-06-30 12:00' WHERE name = 'Desk Lamp';
CREATE VIEW active_products AS SELECT * FROM products WHERE deleted_at IS NULL;
SELECT count(*) AS all_products, (SELECT count(*) FROM active_products) AS active FROM products;
```

Output:

```
 all_products | active
--------------+--------
            9 |      8
(1 row)
```

	Hard delete	Soft delete
Undo / audit	only from backups or audit tables	trivial: clear `deleted_at`
Queries	simple	every query must filter (use a view, or row-level security — per-user rules that hide rows)
Unique constraints	simple	a deleted row still blocks its email — use a partial unique index (PostgreSQL's filtered index) `WHERE deleted_at IS NULL`
Privacy law ("right to be forgotten")	satisfied	must still purge eventually

Deleting millions of rows: do it in batches (e.g. `DELETE … WHERE id IN (SELECT id … LIMIT 10000)` repeated) to keep transactions and locks short; for time-based data, partitioned tables (one logical table stored as several physical pieces, e.g. one per month) let you `DROP` a whole old partition instantly (chapter 15).

Dialect notes: UPDATE / DELETE using another table

	UPDATE	DELETE
PostgreSQL	`UPDATE t SET … FROM s WHERE t.id = s.id`	`DELETE FROM t USING s WHERE t.id = s.id`
SQL Server	`UPDATE t SET … FROM t JOIN s ON …`	`DELETE t FROM t JOIN s ON …`
MySQL	`UPDATE t JOIN s ON … SET t.col = …`	`DELETE t FROM t JOIN s ON …`
SQLite	`UPDATE t SET … FROM s WHERE …` (3.33+)	subquery only (`WHERE id IN (SELECT …)`)

9.11 Upsert: INSERT … ON CONFLICT, and MERGE

What it is

An **upsert** means "insert this row, or if its key already exists, update (or skip) it". `SELECT`-then-`INSERT`-or-`UPDATE` in application code has a race: two sessions both see "not there" and both insert.

PostgreSQL (9.5+) does it atomically (as one indivisible step that no other session can slip in between) with `INSERT … ON CONFLICT`.

How it works

```
INSERT INTO t (cols) VALUES (...)
ON CONFLICT (conflict_target)          -- the unique/PK column(s), or ON CONSTRAINT name
DO NOTHING                             -- skip rows that would conflict
-- or
DO UPDATE SET col = EXCLUDED.col, ...  -- EXCLUDED = the row you tried to insert
          [WHERE condition];           -- update only when this is true
```

1. PostgreSQL tries to insert each row; if it would violate the unique index named by the conflict target, it runs the `DO` action instead of raising an error.
2. In `DO UPDATE`, `t.col` (or an alias) is the **existing** row, `EXCLUDED.col` the **proposed** one.
3. Each row ends up inserted, updated or skipped — atomically, even under concurrency.

```
-- code/11_upsert.sql
CREATE TABLE inventory (
    sku         text PRIMARY KEY,
    qty         integer NOT NULL CHECK (qty >= 0),
    updated_at  timestamp NOT NULL
);
INSERT INTO inventory VALUES ('MOUSE-1', 10, '2025-06-01 08:00'), ('HUB-2', 4, '2025-06-01 08:00');

INSERT INTO inventory VALUES ('MOUSE-1', 99, '2025-06-02 08:00'), ('LAMP-3', 7, '2025-06-02 08:00')
ON CONFLICT (sku) DO NOTHING
RETURNING sku, qty;
```

Output:

```
  sku   | qty
--------+-----
 LAMP-3 |   7
(1 row)
```

MOUSE-1 already existed, so it was skipped silently (and isn't in `RETURNING`).

Insert or add to the existing stock:

```
INSERT INTO inventory AS i VALUES
    ('MOUSE-1', 5, '2025-06-03 08:00'),
    ('KEYB-4',  2, '2025-06-03 08:00')
ON CONFLICT (sku) DO UPDATE
SET qty        = i.qty + EXCLUDED.qty,
    updated_at = EXCLUDED.updated_at
RETURNING sku, qty, updated_at, (xmax <> 0) AS was_update;
```

Output:

```
   sku   | qty |     updated_at      | was_update
---------+-----+---------------------+------------
 MOUSE-1 |  15 | 2025-06-03 08:00:00 | t
 KEYB-4  |   2 | 2025-06-03 08:00:00 | f
(2 rows)
```

(`xmax` is a hidden system column on every PostgreSQL row; `xmax <> 0` is a common trick to tell updated from inserted rows; it relies on an internal detail, so don't build business logic on it.)

Conditional update — only if the incoming data is newer (a common rule when syncing from several sources):

```
INSERT INTO inventory AS i VALUES ('HUB-2', 1, '2025-05-01 00:00')    -- older than what we have
ON CONFLICT (sku) DO UPDATE
SET qty = EXCLUDED.qty, updated_at = EXCLUDED.updated_at
WHERE EXCLUDED.updated_at > i.updated_at
RETURNING sku, qty;
```

Output:

```
 sku | qty
-----+-----
(0 rows)
```

Nothing changed: HUB-2 keeps qty 4 from 1 June (the file lists the table afterwards). **Rules and errors:**

- The conflict target must match a unique index or constraint (`ON CONFLICT (sku)` or `ON CONFLICT ON CONSTRAINT inventory_pkey`). `DO NOTHING` may omit the target (then any unique violation is skipped).
- One statement can't affect the same row twice:

```
INSERT INTO inventory VALUES ('PEN-5', 1, '2025-06-04'), ('PEN-5', 2, '2025-06-04')
ON CONFLICT (sku) DO UPDATE SET qty = EXCLUDED.qty;
```

Output:

```
ERROR:  ON CONFLICT DO UPDATE command cannot affect row a second time
HINT:  Ensure that no rows proposed for insertion within the same command have duplicate constrained values.
```

De-duplicate the input first (e.g. `SELECT DISTINCT ON (sku) …`).

- Other constraint violations (a `CHECK`, a different unique index) still raise errors.

MERGE (PostgreSQL 15+, SQL Server, Oracle)

`MERGE` is the SQL-standard statement for "compare a source with a target and insert, update or delete accordingly". PostgreSQL added it in **version 15** (SQL Server has had it since 2008, Oracle since 9i); this book's server is 14, where `MERGE` is a syntax error. The general shape (PostgreSQL 15+ / SQL Server — not run here):

```
MERGE INTO inventory AS t
USING incoming AS s ON t.sku = s.sku
WHEN MATCHED AND s.qty = 0 THEN DELETE
WHEN MATCHED THEN UPDATE SET qty = s.qty, updated_at = s.updated_at
WHEN NOT MATCHED THEN INSERT (sku, qty, updated_at) VALUES (s.sku, s.qty, s.updated_at);
```

	`INSERT … ON CONFLICT`	`MERGE`
Can delete	no	yes (`WHEN MATCHED … THEN DELETE`)
Needs a unique index	yes	no — any join condition
Safe against concurrent inserts of the same key	yes (that's its purpose)	not by itself: two sessions can both take the `NOT MATCHED` branch → unique violation; retry or lock
`RETURNING`	yes	PostgreSQL 17+ (which also added `WHEN NOT MATCHED BY SOURCE` for deleting target rows missing from the source)

Rule of thumb in PostgreSQL: **upserts of single rows/batches → `ON CONFLICT`; synchronising a table from a staging table with deletes → `MERGE`** (15+).

Dialect notes

	Upsert syntax
PostgreSQL	`INSERT … ON CONFLICT … DO NOTHING / DO UPDATE SET … EXCLUDED.col` (9.5+); `MERGE` (15+)
MySQL	`INSERT … ON DUPLICATE KEY UPDATE col = VALUES(col)` (or row alias `AS new … new.col`, 8.0.19+); `INSERT IGNORE` ≈ do nothing
SQLite	`INSERT … ON CONFLICT(col) DO UPDATE SET col = excluded.col` (3.24+); `INSERT OR IGNORE` / `INSERT OR REPLACE`
SQL Server	`MERGE`, or `UPDATE` + `INSERT … WHERE NOT EXISTS` with locking hints
Oracle	`MERGE`

In the interview

30-second answer: "In PostgreSQL, `INSERT … ON CONFLICT (key) DO UPDATE SET col = EXCLUDED.col` — atomic and concurrency-safe because it relies on the unique index; `DO NOTHING` skips duplicates. `MERGE` (PostgreSQL 15+, SQL Server, Oracle) is the standard multi-branch statement that can also delete, but doesn't by itself protect against concurrent inserts."

Self-check

1. *What does `EXCLUDED` refer to?* → The row that was proposed for insertion.
2. *What does `ON CONFLICT` need?* → A unique index/constraint matching the conflict target.

9.12 Backup and restore

What it is

A **backup** is a copy you can rebuild the database from; a **restore** does the rebuilding. Backups protect against what replication (keeping live copies — **replicas** — of the database on other servers; see System Design Fundamentals) can't: a mistaken `DELETE`/`DROP` (replicas copy it instantly), data-corrupting bugs, ransomware (malware that encrypts your files and demands payment), losing a data centre.

Two terms decide a backup strategy:

- **RPO — recovery point objective**: how much recent data you can afford to lose (e.g. 5 minutes).
- **RTO — recovery time objective**: how long you can afford to be down while restoring (e.g. 1 hour).

Kinds of backups

Kind	What is copied	Tools (PostgreSQL)	Good for	Limits
Logical	SQL / an archive of schema + data	`pg_dump`, `pg_dumpall`	small/medium DBs, single tables, moving across versions	slow for huge DBs; restore rebuilds indexes
Physical	the database files themselves	`pg_basebackup`, file-system snapshots, pgBackRest, Barman	large DBs, fast full restore	whole cluster (in PostgreSQL: one server instance with all its databases) only; same major version (e.g. 16.x)
Continuous (PITR)	a base backup **plus** the write-ahead log (WAL) stream	WAL archiving + base backup	restore to *any moment* (just before the bad `DELETE`)	more setup and storage

> **Key term — WAL and PITR:** the **write-ahead log (WAL)** is a file of every change, written *before* the change is applied to the tables, so the database can replay it after a crash. **Point-in-time recovery (PITR)** restores a **base backup** (a full physical copy) and then replays archived WAL up to any chosen moment — for example one second before a bad `DELETE`.

Other terms: **full** (everything), **differential** (changes since the last full), **incremental** (changes since the last backup of any kind; PostgreSQL 17 added incremental `pg_basebackup`), **hot/online** (taken while the database is in use — all of the above are).

pg_dump and pg_restore, for real

`pg_dump` takes a **consistent** snapshot of one database while others keep reading and writing (it runs inside a single repeatable-read transaction — an isolation level where the transaction sees the data exactly as it was when it started; chapter 13). Formats:

Format flag	Output	Restore with	Notes
`-F p` (plain, default)	a `.sql` script	`psql -f`	readable, editable; replayed statement by statement (`--single-transaction` for all-or-nothing)
`-F c` (custom)	one compressed archive	`pg_restore`	pick objects to restore, reorder, parallel restore (`-j`)
`-F d` (directory)	a folder, one file per table	`pg_restore`	the only format that can **dump** in parallel (`-j`)
`-F t` (tar)	a tar archive (a standard Unix file bundle)	`pg_restore`	like custom, but no compression or parallel restore

The script does a full round trip on the book's private server, in a temporary directory deleted afterwards:

```
# code/12_backup_restore.sh (key lines; CONN="-h /tmp -p 5544 -U postgres")
pg_dump $CONN -d shop -F p -f "$WORK/shop.sql"          # plain SQL script
pg_dump $CONN -d shop -F c -f "$WORK/shop.dump"         # custom archive
pg_dump $CONN -d shop -F d -j 2 -f "$WORK/shop_dir"     # directory format, 2 parallel jobs
pg_restore -l "$WORK/shop.dump"                         # list the archive's contents
psql $CONN -d shop -c "DROP TABLE order_items"          # the "disaster"
pg_restore $CONN -d shop -t order_items "$WORK/shop.dump"      # bring back one table
pg_restore $CONN -d shop_copy -j 2 "$WORK/shop.dump"    # full restore into a new, empty DB
psql $CONN -d shop_plain -f "$WORK/shop.sql"            # plain dumps are replayed by psql
pg_dump $CONN -d shop -t products --schema-only         # structure only (--data-only: rows only)
```

Output (run with `bash chapters/09-ddl-dml-and-constraints/code/12_backup_restore.sh` from the book root):

```
== 1. a database to protect
shop: 10 customers, 14 orders, 22 order_items
== 2. logical backups in three formats
...
-- first CREATE TABLE in the plain dump:
CREATE TABLE public.customers (
    customer_id integer NOT NULL,
    first_name text NOT NULL,
...
-- plain dump contains 8 COPY blocks (one per table with data)
== 3. what is inside the custom-format archive (pg_restore -l = table of contents)
TABLE public customers postgres
TABLE public orders postgres
TABLE DATA public customers postgres
...
== 4. disaster: someone drops a table
ERROR:  relation "order_items" does not exist
== 5. restore just that table (definition + data) from the archive
shop: 10 customers, 14 orders, 22 order_items
-- note: -t restores the table and its rows, not its PK/FK constraints:
0 constraints on order_items
== 6. full restore into a NEW database, 2 parallel jobs
shop_copy: 10 customers, 14 orders, 22 order_items
3 constraints on order_items
== 7. a plain dump is restored with psql
shop_plain: 10 customers, 14 orders, 22 order_items
== 8. schema-only and data-only dumps of one table
CREATE TABLE statements in schema-only dump: 1, in data-only dump: 0
...
```

What it shows:

1. A plain dump is just SQL: `CREATE TABLE`s, a `COPY … FROM stdin` block of rows per table, then constraints and indexes (added *after* the data, which is faster). `stdin` means *standard input*: the rows follow inline in the same script.
2. A custom archive has a **table of contents** (`pg_restore -l`); `-t`, `-n` (schema) and `-L` (edited list) choose what to restore.
3. `-t` restores a table's definition and rows **but not its constraints or indexes** (separate archive entries) — re-add them with `-L` or by re-running the DDL.

Useful options: `--clean --if-exists` (drop objects before recreating), `--create` (create the database itself), `--no-owner` (when restoring as a different user), `-j N` (parallelism). `pg_dump` dumps one database; **roles and tablespaces** (database users/groups, and named disk locations for data) are cluster-wide, so also run `pg_dumpall --globals-only`.

Backup and restore in other databases

	Full backup	Restore	Notes
PostgreSQL	`pg_dump` (logical), `pg_basebackup` + WAL (physical, PITR)	`pg_restore` / `psql`; restore base backup + replay WAL to a target time	command-line tools, not SQL statements
MySQL	`mysqldump`, MySQL Shell dump utilities (logical); Percona XtraBackup / MySQL Enterprise Backup (physical)	`mysql < dump.sql`; binary logs (MySQL's log of every change) for point-in-time	
SQL Server	`BACKUP DATABASE db TO DISK = '…\db.bak'` (also `WITH DIFFERENTIAL`, `BACKUP LOG`, `TO URL` for cloud storage)	`RESTORE DATABASE db FROM DISK = '…' [WITH NORECOVERY / RECOVERY, STOPAT = …]`	backup and restore are T-SQL (SQL Server's SQL dialect) statements; the others mostly use separate tools
SQLite	`.backup file` in the shell, `VACUUM INTO 'file'`	copy the file back / `.restore`	the database is one file

Two corrections to things some tutorials show (SQL Server — not run):

- `BACKUP DATABASE` has no `WITH SCHEDULE = …` option. Backups are scheduled outside the statement — SQL Server Agent jobs or maintenance plans (elsewhere: cron or systemd timers — the standard Linux job schedulers — or a backup tool's scheduler).
- There is no `RESTORE TABLE`. To recover one table, restore the backup under **another database name** (`RESTORE DATABASE db_old FROM DISK = … WITH MOVE …`) and copy the table across — like our `pg_restore -t` or `shop_copy`.

Best practices

- **3-2-1 rule**: 3 copies, on 2 kinds of storage, 1 off-site.
- **Test restores regularly** — an untested backup is a hope. Automate a restore into a scratch server and run sanity queries (row counts, as the script does).
- **Encrypt** backups and restrict who can read them.
- **Match RPO/RTO**: nightly `pg_dump` alone risks 24 h of loss; PITR gets it to seconds.
- **Replication is not a backup** — replicas faithfully copy `DROP TABLE` (chapter 15).
- Managed databases (run for you by a cloud provider: Amazon RDS, Cloud SQL, Azure) provide automated snapshots + PITR within a retention window (how long backups are kept).

In the interview

> **30-second answer:** "Logical backups (`pg_dump`) export schema and data — flexible, single tables, portable across versions, slow for huge databases. Physical backups copy the data files; with archived WAL they allow point-in-time recovery. Choose by RPO/RTO, keep 3-2-1 copies, encrypt, test restores — and replication doesn't protect against a bad `DELETE`."

- *Someone ran `DELETE` without `WHERE` at 14:03 — how do you recover?* → PITR: restore the last base backup to a new server and replay WAL up to 14:02:59, then copy the lost rows back (or fail over to it — switch the application to use it).
- *pg_dump on a busy database — does it block?* → No; it reads a consistent snapshot. It does take light locks that block DDL like `DROP` or `ALTER` on dumped tables while it runs.

Self-check

1. *Which `pg_dump` formats can `pg_restore` read?* → custom, directory and tar (plain is replayed with `psql`).
2. *Which format can dump in parallel?* → Directory (`-F d -j N`).
3. *What does `pg_dump` not include that `pg_dumpall --globals-only` does?* → Roles and tablespaces.

Chapter 9 TL;DR

- **Types** validate as well as store: `numeric` for money, `integer`/`bigint` for ids, `text`, `timestamptz`, `date`, `boolean`.
- `CREATE` databases, schemas, tables (`IF NOT EXISTS`, `AS SELECT` = data without constraints, `LIKE … INCLUDING ALL` = structure, `TEMP`).
- **Keys**: super ⊇ candidate → one primary (unique + not null) + alternate keys (`UNIQUE`); composite keys; foreign keys with `ON DELETE CASCADE | SET NULL | RESTRICT | NO ACTION`. `CHECK` rejects only false; `UNIQUE` allows several NULLs (one in SQL Server). Name constraints; index FK columns.
- **Generated keys**: `GENERATED ALWAYS AS IDENTITY` (not `serial`), sequences have gaps, UUIDs for distributed ids, `GENERATED … STORED` for derived columns.
- `ALTER`: add/drop/rename columns, `TYPE … USING`, `SET NOT NULL` after backfilling, `NOT VALID` + `VALIDATE` on big tables; watch exclusive locks.
- `DROP` removes the object, `TRUNCATE` empties it fast (transactional in PostgreSQL, `RESTART IDENTITY`, `CASCADE`), `DELETE` removes chosen rows.
- **Indexes**: `CREATE [UNIQUE] INDEX [CONCURRENTLY]`; clustered vs non-clustered; details in chapter 12.

- **DML**: `INSERT` (list columns, multi-row, `INSERT … SELECT`), `UPDATE` (`SET` sees old values, `FROM`), `DELETE` (`USING`, children first), all with `RETURNING`. Practise destructive statements in `BEGIN … ROLLBACK`.
- **Upsert**: `INSERT … ON CONFLICT … DO NOTHING / DO UPDATE SET … = EXCLUDED.…`; `MERGE` needs PostgreSQL 15+.
- **Backups**: `pg_dump` (plain/custom/directory) + `pg_restore` (selective, parallel); physical + WAL for PITR; RPO/RTO, 3-2-1, test restores.

Next: chapter 10 — data modeling and ER diagrams, which decides *which* tables and keys to create in the first place.